EXPERIMENT-3
SANJAY DHAKAR
(24UADS1046)

Objective :- WAP to implement a three-layer neural network using Tensor flow library (only, no keras) to classify MNIST handwritten digits dataset. Demonstrate the implementation of feed-forward and back-propagation approaches.

Description :-

A BPNN is a multi-layer feed-forward neural network trained using the Backpropagation algorithm.

It has:

- Input Layer
- One or more Hidden Layers
- Output Layer

In your case → Three-layer neural network

- Layer 1: Input (784 neurons for MNIST 28×28)
- Layer 2: Hidden layer
- Layer 3: Output layer (10 neurons for digits 0–9)

WORKING MECHANISM :-

🔁 Working Mechanism

Step 1: Feed Forward

Each layer computes:

$$Z = WX + b$$
$$A = f(Z)$$

Where:

- WWW = weights
- bbb = bias
- fff = activation function

For MNIST:

- Hidden → ReLU
- Output → Softmax

Softmax gives probability distribution:

$$Softmax(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Step 2: Loss Calculation

We use Cross Entropy Loss :

$$L = -\sum y \log(\hat{y})$$

Where:

- y= true label
- y^ = predicted probability

Step 3: Backpropagation

Using chain rule:

$$\frac{\partial L}{\partial W}$$

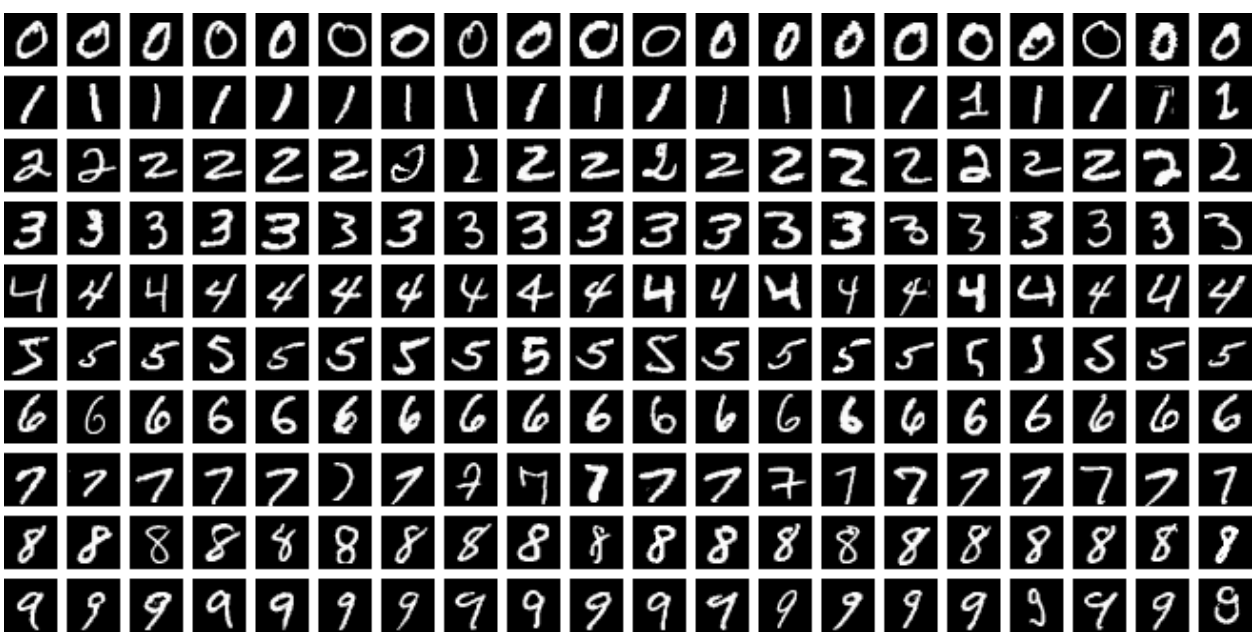
TensorFlow automatically computes gradients using : tf.GradientTape()

Then weights updated:

$$W = W - \eta \frac{\partial L}{\partial W}$$

Where η = learning rate.

📊 MNIST Dataset



- 60,000 training images
- 10,000 test images
- 28×28 grayscale images
- 10 classes (0–9)

Source Code :-

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Load MNIST dataset from tf.keras.datasets (only for data loading)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train.reshape(-1, 784).astype(np.float32) / 255.0
x_test = x_test.reshape(-1, 784).astype(np.float32) / 255.0

y_train = tf.one_hot(y_train, 10)
y_test = tf.one_hot(y_test, 10)

# Hyperparameters
learning_rate = 0.01
epochs = 50
batch_size = 128

# Initialize weights and biases
W1 = tf.Variable(tf.random.normal([784, 128], stddev=0.1))
b1 = tf.Variable(tf.zeros([128]))

W2 = tf.Variable(tf.random.normal([128, 10], stddev=0.1))
b2 = tf.Variable(tf.zeros([10]))

optimizer = tf.optimizers.SGD(learning_rate)

train_loss_list = []
train_acc_list = []

# Training Loop
```

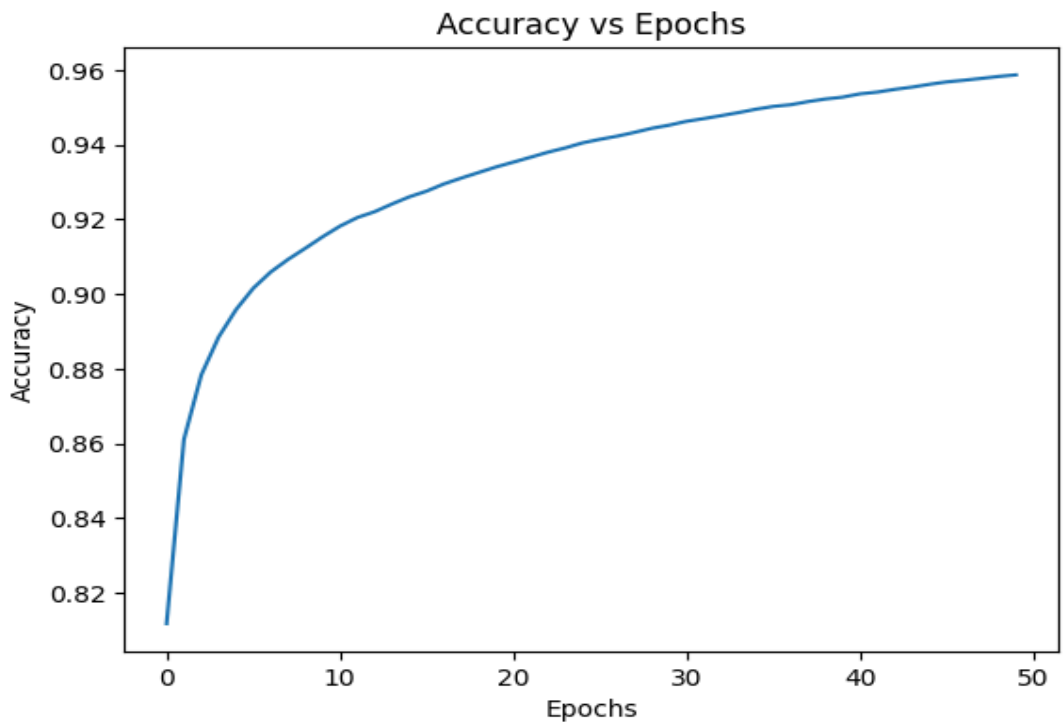
```python
for epoch in range(epochs):

    for i in range(0, x_train.shape[0], batch_size):
        x_batch = x_train[i:i+batch_size]
        y_batch = y_train[i:i+batch_size]

        with tf.GradientTape() as tape:

            # Feed Forward
            z1 = tf.matmul(x_batch, W1) + b1
            a1 = tf.nn.relu(z1)

            z2 = tf.matmul(a1, W2) + b2
            y_pred = tf.nn.softmax(z2)

            # Loss
            loss = tf.reduce_mean(
                tf.nn.softmax_cross_entropy_with_logits(
                    labels=y_batch,
                    logits=z2
                )
            )

        # Backpropagation
        gradients = tape.gradient(loss, [W1, b1, W2, b2])
        optimizer.apply_gradients(zip(gradients, [W1, b1, W2, b2]))

    # Accuracy calculation
    z1 = tf.matmul(x_train, W1) + b1
    a1 = tf.nn.relu(z1)
    z2 = tf.matmul(a1, W2) + b2
    y_pred = tf.nn.softmax(z2)

    correct = tf.equal(tf.argmax(y_pred, 1), tf.argmax(y_train, 1))
    accuracy = tf.reduce_mean(tf.cast(correct, tf.float32))

    train_loss_list.append(loss.numpy())
    train_acc_list.append(accuracy.numpy())

    if epoch % 5 == 0:
```

```
        print(f"Epoch {epoch}: Loss = {loss.numpy():.4f}, Accuracy =
{accuracy.numpy():.4f}")

# Final Test Accuracy
z1 = tf.matmul(x_test, W1) + b1
a1 = tf.nn.relu(z1)
z2 = tf.matmul(a1, W2) + b2
y_pred_test = tf.nn.softmax(z2)

correct = tf.equal(tf.argmax(y_pred_test, 1), tf.argmax(y_test, 1))
test_accuracy = tf.reduce_mean(tf.cast(correct, tf.float32))

print("\nFinal Test Accuracy:", test_accuracy.numpy())

plt.figure()
plt.plot(train_loss_list)
plt.title("Loss vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()

plt.figure()
plt.plot(train_acc_list)
plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.show()
```

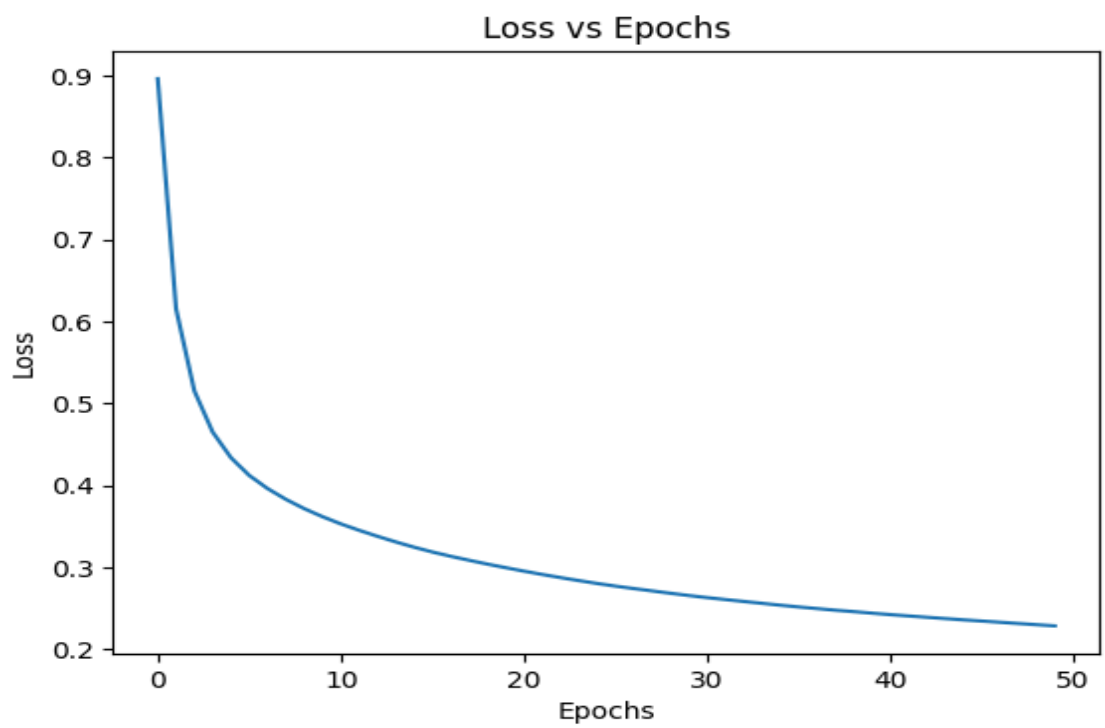
Output:-

```
Epoch 0: Loss = 0.8963, Accuracy = 0.8117
Epoch 5: Loss = 0.4123, Accuracy = 0.9016
Epoch 10: Loss = 0.3532, Accuracy = 0.9182
Epoch 15: Loss = 0.3189, Accuracy = 0.9276
Epoch 20: Loss = 0.2956, Accuracy = 0.9354
Epoch 25: Loss = 0.2773, Accuracy = 0.9415
Epoch 30: Loss = 0.2633, Accuracy = 0.9463
Epoch 35: Loss = 0.2519, Accuracy = 0.9503
Epoch 40: Loss = 0.2425, Accuracy = 0.9536
Epoch 45: Loss = 0.2347, Accuracy = 0.9568
Final Test Accuracy: 0.9563
```

GRAPHS :


Accuracy vs Epochs
Accuracy
Epochs
0.96
0.94
0.92
0.90
0.88
0.86
0.84
0.82
0
10
20
30
40
50

-


Loss vs Epochs
Loss
Epochs
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0
10
20
30
40
50

MY COMMENTS :-

From this experiment related to BPNN(Backpropagation Neural Network) , I understand the key concept in neural network training i.e forward propagation and back propagation . I got to know how this algorithm works mathematically and also , how backpropagation works intuitively .

The key takeaways from this experiment are : -
1 . Mathematical reasoning of gradients and derivatives used in neural network
2 . How the Backpropagation works intuitively
3. How simple intuition of solving the derivatives can save use so much compute power